

DICCIONARIOS

Operaciones

- `pair<it, bool> insert(<clave, valor> const&)` : si existe devuelve false, si no lo añade y devuelve true. En la LIBRERIA devuelve un `it` al elemento insertado y el booleano.
- `int count(clave const&)` : devuelve 1 si pertenece, 0 si no pertenece.
- `it find(clave const&)` : devuelve `it` al elemento si existe o `it` al final de la tabla si no.
- `valor const& at (clave const&)` : devuelve el valor. SOLO CONSULTA.
- `valor & operator []` : permite consultar y modificar el valor de la clase, si no existe la inserta.
- `erase (clave const&)` : borra la clave, si no existe devuelve false.
- `empty () const` : devuelve booleano con si esta vacío o no.
- `int size()` : devuelve el nº de elementos en el dicc.

MAP (ordenado)

Se implementa con un árbol binario de búsqueda.

Las operaciones de **añadir**, **buscar**, **borrar** un elemento tienen coste **logarítmico**.

La operación de **crear** tiene un coste **constante**.

Toda la memoria reservada está ocupada. **No** se **redimensiona** cuando están completos.

UNORDERED_MAP (no orden)

Se implementa con una tabla hash.

Las operaciones de **añadir**, **buscar**, **borrar** un elemento tienen coste **constante** en el nº de elementos del diccionario.

La operación de **crear** tiene un coste **lineal** en el tamaño de la tabla.

No ocupan toda la memoria reservada. Se **redimensiona** cuando se alcanza el factor de carga.

TABLAS HASH

Factor de carga = nº valores / tamaño de la tabla (siempre <1 , se recomienda entre 0'7 y 0'8)

El tamaño de la tabla tiene que ser PRIMO, si la fórmula nos da un nº no primo, cogemos el siguiente primo a ese.

Cada clave se convierte en un índice del vector con una **función de dispersión**. Para calcular la pos de la clave en la tabla: **clave % tamaño (func hash)**. Se produce una colisión si dos claves tienen el mismo hash.

Abierta

El tamaño de la tabla dependerá del factor de carga que queramos. Se implementa como: una clave y una lista con los valores de esa clave, como no hay orden da igual cómo añadir los valores en la lista.

Si hay una colisión, añado el valor de la clave a la lista (por delante).

¿Cómo redimensionarla? se reserva memoria para el nuevo vector con tamaño el 1º primo mayor al doble del tamaño act. Se recorre la tabla calculando el nuevo hash y no se copian los valores si no que se enlaza la lista a la nueva mem. Coste = a la cerrada.

Cerrada

Ya no se implementa con listas, si no que es un único vector con pares <clave, valor>.

Ahora las colisiones hay que tratarlas diferentes ya que no tenemos la lista:

cundo una clave colisiona con otra, la clave sinónima se coloca en la siguiente posición libre desde la q correspondería.

Pero, ahora para buscar un valor es distinto: accedo a la componente que me da el hash (que es donde debería estar mi valor) si el valor que busco es ese he acabado, si no la búsqueda termina cuando encontramos una posición vacía, ya que a partir de ahí no podría estar porque se hubiera insertado ahí.

Problema al eliminar: dejaríamos huecos intermedios y la cagaríamos para la búsqueda, así que se añade otro campo que es el estado del valor: libre, ocupada y liberada. La liberada se considera como vacía ya que es como si se hubiera "eliminado" de la tabla.

¿Cómo redimensionarla? Se reserva memoria para un vector con tamaño el 1º primo mayor que el doble del tamaño de la tabla. Se recorre la tabla original calculando el nuevo hash y colocando los elementos en su nueva pos. El coste es $O(\max(k, N))$ siendo k el coste de reservar memoria para la nueva tabla y N el nº de elementos de la tabla original. Como el nº de elems de la nueva tabla es $> q$ el de la original $\rightarrow$ el máx será el nº de elems de la nueva tabla.

